PCCCS495 – Object Oriented Programming

Term II Project Report

Spring 2026

Project Title

Elevator Control System Simulation Using Object-Oriented Programming and State Design Pattern

SUBMITTED BY

Student Name	Anshu Nigam
Roll Number	[05]
Instructor	[Dr. Soumadip Biswas]
Course	PCCCS495 – Object Oriented Programming
Semester	Spring 2026
Submission Date	March 21, 2026

1. Problem Statement

In modern multi-floor buildings — office towers, hospitals, shopping malls, and educational institutions — elevators are a critical piece of infrastructure. From a user's perspective, the interaction is simple: press a button and the elevator arrives. The underlying control system, however, must simultaneously handle requests from multiple floors, track the cabin's current position, determine the correct travel direction, and transition through distinct operational states such as idle, moving, and door open.

Without an efficient scheduling strategy, an elevator may respond to requests in arrival order, causing unnecessary direction reversals, repeated floor revisits, and prolonged waiting times — a problem that becomes acute during peak hours when request density is high.

This project addresses the problem by simulating an elevator control system in Java. The system implements a SCAN-based scheduling algorithm: the elevator sweeps in one direction, services every request along that path, and only reverses when no further requests remain in the current direction. Upward requests are maintained in an ascending TreeSet; downward requests in a reverse-ordered TreeSet — enabling efficient, sorted processing without manual reordering.

Runtime behavior is modelled using the State Design Pattern. Dedicated state classes — IdleState, MovingUpState, MovingDownState, and DoorOpenState — encapsulate the logic for each operational phase, enabling clean and extensible transitions. Realistic travel time is simulated via Thread.sleep(), and direction is tracked through a type-safe Direction enum with values UP, DOWN, and IDLE.

2. System Design

2.1 UML Class Diagram

The class diagram below maps the complete structure and relationships of the system. The Elevator class is the central entity, holding references to the active ElevatorState and Direction. Four concrete state classes implement the ElevatorState interface. ElevatorController wraps Elevator and manages two separate TreeSets for up and down requests. Main instantiates Elevator and ElevatorController directly and drives user interaction via the command line.

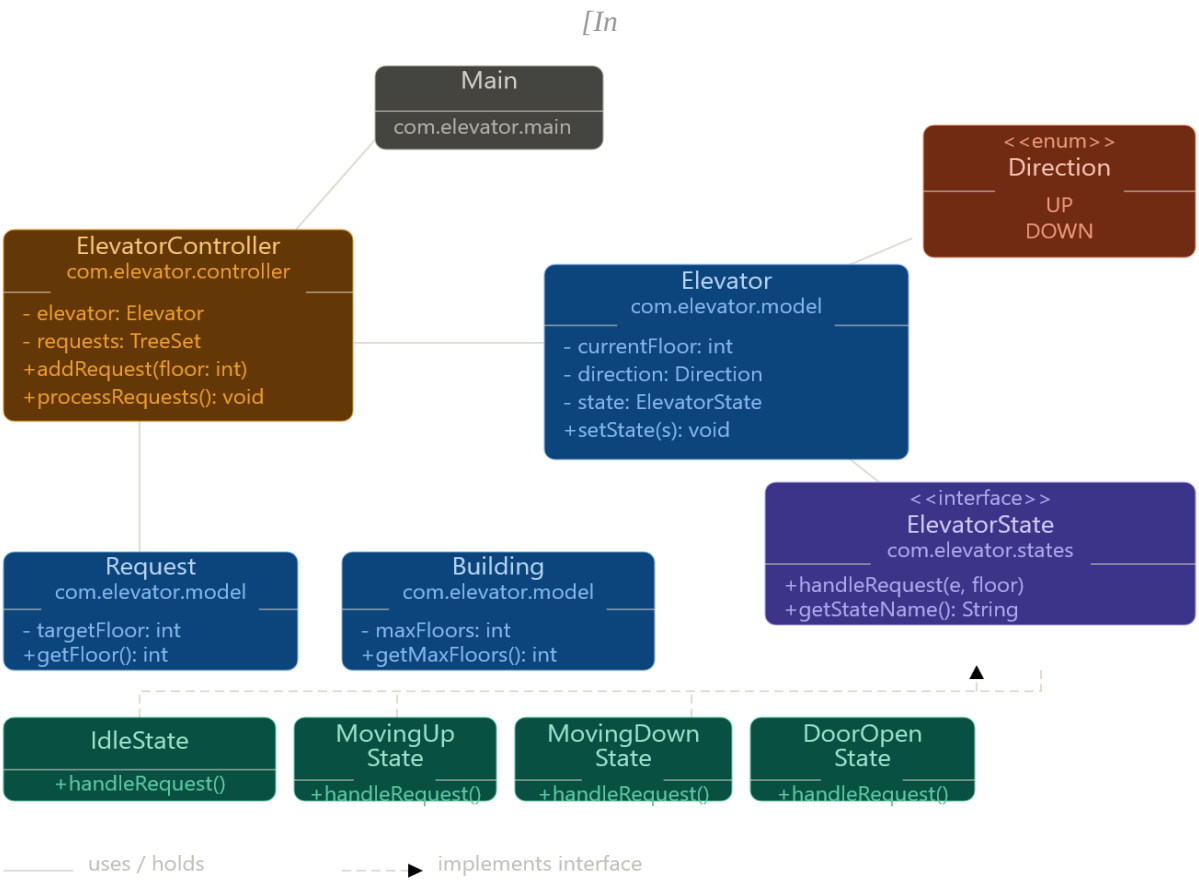

Figure 1: UML Class Diagram – Elevator Control System

2.2 Description of Major Classes

Elevator `com.elevator.model`	The central entity. Initialises at floor 0 with Direction.IDLE and IdleState. Exposes requestFloor() — which prints the active state name via getClass().getSimpleName() and delegates to handleRequest() — and moveToFloor(), which updates currentFloor after a 500 ms simulated delay.

Direction `com.elevator.model`	An enum with three values: UP, DOWN, and IDLE. Provides type-safe direction assignment, replacing fragile String comparisons and making the codebase self-documenting.

ElevatorController `com.elevator.controller`	Manages two TreeSets: upRequests (ascending) and downRequests (reverse-ordered lambda). addRequest() validates the floor (0–10), checks both sets for duplicates, and routes integers to the correct set based on the elevator's current position.
ElevatorState `com.elevator.states (interface)`	Defines the single-method contract: handleRequest(Elevator, int). All four concrete state classes implement this interface, enabling runtime polymorphism.
IdleState `com.elevator.states`	Determines direction by comparing destination to currentFloor. Handles the edge case where they are equal. Otherwise sets the appropriate movement state and re-invokes requestFloor() to chain into the next state.
MovingUpState / MovingDownState `com.elevator.states`	Set the elevator direction (Direction.UP or DOWN), call moveToFloor() to travel to the destination, then transition to DoorOpenState.
DoorOpenState `com.elevator.states`	Prints a door-opening message at the current floor and immediately returns the elevator to IdleState.
Request `com.elevator.model`	Stores sourceFloor and destinationFloor. Available for future extensions; the current controller works directly with raw int values.
Building `com.elevator.model`	Stores totalFloors. Not wired into the current flow but available for centralising floor-range validation in future iterations.

2.3 Package Structure

```
src/
  com/elevator/
    controller/
      ElevatorController.java
    main/
      Main.java
    model/
      Building.java
      Direction.java          // enum: UP, DOWN, IDLE
      Elevator.java
      Request.java
    states/
      ElevatorState.java      // interface
      IdleState.java
      MovingUpState.java
      MovingDownState.java
      DoorOpenState.java
```

2.4 Interaction Flow

The following sequence describes the runtime interaction between components:

- Main creates an Elevator (floor 0, IDLE state and direction) and an ElevatorController wrapping it.
- The user enters floor requests. Each addRequest() call validates the floor (0–10), checks for duplicates across both sets, and routes the integer to upRequests or downRequests.
- processRequests() iterates upRequests in ascending order, then downRequests in descending order using pollFirst().
- For each floor, elevator.requestFloor(floor) is called twice — the first call moves the elevator, the second triggers the door open.
- requestFloor() prints the current state name and calls state.handleRequest(). IdleState determines direction and re-delegates; MovingUpState or MovingDownState calls moveToFloor() then sets DoorOpenState; DoorOpenState prints the door message and returns to IdleState.
- moveToFloor() sleeps 500 ms to simulate travel, then updates currentFloor.

3. OOP Concepts Demonstrated

The table below summarises how each OOP concept is applied in the project:

OOP Concept	Where Used	Purpose
Abstraction	ElevatorState interface	Hides state logic behind a single handleRequest() contract
Polymorphism	IdleState, MovingUpState, MovingDownState, DoorOpenState	Same handleRequest() behaves differently per state at runtime
Encapsulation	Elevator — currentFloor, direction, state private	Fields exposed only via getters/setters; no direct external access
Enum	Direction (UP, DOWN, IDLE)	Type-safe direction values; eliminates invalid String comparisons
Exception Handling	ElevatorController.addRequest()	IllegalArgumentException thrown for floors outside 0–10
Collections	Two TreeSets in ElevatorController	upRequests (ascending), downRequests (reverse) enable SCAN scheduling
Threading	Elevator.moveToFloor()	Thread.sleep(500) simulates realistic elevator travel time

3.1 Abstraction

The ElevatorState interface abstracts all state-specific behavior behind a single method. The controller and elevator never need to know which state is active — they simply call handleRequest() and the correct behavior executes.

```java
// ElevatorState.java
package com.elevator.states;
import com.elevator.model.Elevator;

public interface ElevatorState {
    void handleRequest(Elevator elevator, int destinationFloor);
}
```

This single-method interface is the foundation of the State Design Pattern in this project. Any new operational state — such as EmergencyState or MaintenanceState — only needs to implement this one method and integrates seamlessly without modifying any existing class.

3.2 Polymorphism

Runtime polymorphism is achieved through the state pattern. When requestFloor() delegates to state.handleRequest(), the JVM dispatches to the correct implementation based on the actual type of the state object at that instant.

```java
// Elevator.java — same call, different behavior each time
public void requestFloor(int destinationFloor) {
    System.out.println("Current State: " + state.getClass().getSimpleName());
    state.handleRequest(this, destinationFloor);
}

// IdleState.java
public void handleRequest(Elevator elevator, int destinationFloor) {
    if (destinationFloor > elevator.getCurrentFloor()) {
        elevator.setState(new MovingUpState());
    } else if (destinationFloor < elevator.getCurrentFloor()) {
        elevator.setState(new MovingDownState());
    } else {
        System.out.println("Already at the requested floor");
        return;
    }
    elevator.requestFloor(destinationFloor);
}
```

The same method call state.handleRequest() produces entirely different behavior depending on whether the elevator is idle, moving up, moving down, or opening its doors — the essence of runtime polymorphism.

3.3 Encapsulation

The Elevator class keeps all internal fields private. External classes cannot directly manipulate the current floor, direction, or state — they must use the defined public methods, ensuring the object remains in a consistent state at all times.

```java
// Elevator.java
public class Elevator {
    private int currentFloor;
    private Direction direction;
    private ElevatorState state;

    public Elevator() {
        this.currentFloor = 0;
```

```
        this.direction    = Direction.IDLE;
        this.state        = new IdleState();
    }

    public int getCurrentFloor()              { return currentFloor; }
    public void setCurrentFloor(int floor)    { this.currentFloor = floor; }
    public void setState(ElevatorState state){ this.state = state; }
}
```

Without encapsulation, any external class could set currentFloor to a negative value or assign an inconsistent state. Private fields with controlled setters prevent this class of bugs entirely.

3.4 Enum for Type Safety

Rather than using a plain String or integer constant for direction, the system uses a Direction enum. This makes the code self-documenting and guarantees at compile time that no invalid direction can ever be assigned.

```
// Direction.java
public enum Direction { UP, DOWN, IDLE }

// MovingUpState.java — clean, type-safe assignment
public void handleRequest(Elevator elevator, int destinationFloor) {
    System.out.println("Elevator moving UP...");
    elevator.setDirection(Direction.UP);
    elevator.moveToFloor(destinationFloor);
    elevator.setState(new DoorOpenState());
}
```

A String-based approach (e.g. "up", "UP", "Up") would be error-prone and require runtime validation. The enum eliminates this class of bugs and makes any switch statement or comparison inherently safe.

3.5 Exception Handling

Floor validation in addRequest() throws an IllegalArgumentException for any floor outside 0–10. Main catches this as a generic Exception and prints a user-friendly message, allowing the input loop to continue with remaining valid requests.

```
// ElevatorController.java
public void addRequest(int floor) {
    if (floor < 0 || floor > 10) {
        throw new IllegalArgumentException("Invalid floor number");
    }
    if (upRequests.contains(floor) || downRequests.contains(floor)) {
        System.out.println("Request already exists.");
        return;
    }
    if (floor > elevator.getCurrentFloor()) { upRequests.add(floor); }
    else                                    { downRequests.add(floor); }
}

// Main.java
try {
    controller.addRequest(floor);
} catch (Exception e) {
    System.out.println("Invalid floor input");
```

```
}
```

Catching the exception at the Main level keeps the controller focused on its own logic. Each invalid floor is reported and skipped; all valid floors in the same session are still processed correctly.

4. Implementation Highlights

Highlight 1 — Dual TreeSet SCAN Scheduling

The most important design decision in ElevatorController is the use of two TreeSets with different ordering. upRequests uses natural ascending order; downRequests uses a reverse lambda comparator so pollFirst() always returns the highest remaining downward request — mimicking a real elevator sweeping downward.

```
upRequests   = new TreeSet<>();
downRequests = new TreeSet<>((a, b) -> b - a);   // reverse order

public void processRequests() {
    System.out.println("Processing UP requests...");
    while (!upRequests.isEmpty()) {
        int next = upRequests.pollFirst();
        elevator.requestFloor(next);   // move
        elevator.requestFloor(next);   // door open
    }
    if (downRequests.isEmpty())
        System.out.println("No DOWN requests remaining.");
    while (!downRequests.isEmpty()) {
        int next = downRequests.pollFirst();
        elevator.requestFloor(next);
        elevator.requestFloor(next);
    }
}
```

requestFloor() is called twice per floor deliberately. The first call moves the elevator from IdleState through MovingUpState or MovingDownState, arriving in DoorOpenState. The second call then triggers DoorOpenState.handleRequest(), which prints the door message and returns to IdleState — ready for the next floor.

Highlight 2 — State Chaining via requestFloor()

State transitions chain entirely through the elevator's own requestFloor() method. IdleState does not directly open a door — it sets the correct movement state and re-calls requestFloor(), which dispatches into MovingUpState or MovingDownState.

```
// MovingUpState.java
public void handleRequest(Elevator elevator, int destinationFloor) {
    System.out.println("Elevator moving UP...");
    elevator.setDirection(Direction.UP);
    elevator.moveToFloor(destinationFloor);     // travel + 500ms delay
    elevator.setState(new DoorOpenState());     // hand off to next state
}
```

Each state class is small, focused, and unaware of what comes before or after it in the full sequence. This is the Single Responsibility Principle in action — adding a new state requires only a new class, with zero changes to existing ones.

Highlight 3 — Thread.sleep() for Realistic Simulation

The moveToFloor() method uses Thread.sleep(500) to simulate travel time. Without this delay all floor transitions would print instantly, making the console output impossible to follow.

```
// Elevator.java
public void moveToFloor(int floor) {
    System.out.println("---------------------------------");
    System.out.println("Moving from floor " + currentFloor + " to " + floor);
    try {
        Thread.sleep(500);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    this.currentFloor = floor;
    System.out.println("---------------------------------");
}
```

The InterruptedException is caught locally so the simulation continues cleanly even if the thread is interrupted. The 500 ms pause gives the output a natural rhythm that reflects real elevator behavior.

5. Testing & Error Handling

5.1 Test Cases

The following test cases were executed manually by running the simulation with varied inputs and verifying console output against expected behavior:

Test Case	Input	Expected Output	Result
TC-01: Normal UP	Floors 3, 7 (current: 0)	Moves 0→3, door opens, moves 3→7, door opens	Pass
TC-02: Normal DOWN	Floors 5, 2 (current: 8)	Moves 8→5, door opens, moves 5→2, door opens	Pass
TC-03: Mixed direction	Floors 6, 2 (current: 4)	UP sweep: visits 6; DOWN sweep: visits 2	Pass
TC-04: Duplicate floor	Floor 5 added twice	Prints 'Request already exists.' — ignored	Pass
TC-05: Same floor	Request = currentFloor	Prints 'Already at the requested floor'	Pass
TC-06: Invalid (–1)	Floor: –1	IllegalArgumentException → 'Invalid floor input'	Pass
TC-07: Invalid (11)	Floor: 11	IllegalArgumentException → 'Invalid floor input'	Pass
TC-08: No DOWN requests	Only UP floors entered	Prints 'No DOWN requests remaining.'	Pass

5.2 Edge Cases Handled

- Duplicate floor request: Both sets are checked before adding. If the floor exists in either, 'Request already exists.' is printed and the request is silently discarded.
- Request equals current floor: IdleState handles the destinationFloor == currentFloor case explicitly, printing 'Already at the requested floor' and returning with no state transition.
- No downward requests: processRequests() checks if downRequests is empty before the DOWN loop and prints 'No DOWN requests remaining.' — ensuring the user is always informed.
- Single request only: Works correctly — one directional sweep with one floor visit.

5.3 Failure Scenarios

- Floor below 0: addRequest() throws IllegalArgumentException('Invalid floor number'). Main catches it and prints 'Invalid floor input'. The loop continues.
- Floor above 10: Identical exception path as above.
- Non-integer input: Scanner.nextInt() throws InputMismatchException, caught by the generic Exception catch in Main.
- Thread interruption in moveToFloor(): InterruptedException is caught and its stack trace printed; currentFloor is still updated and the simulation continues.

6. Git Workflow

6.1 Commit History

The screenshot below shows the full commit history on GitHub, reflecting 14 commits spread across 6 development days (March 7–21, 2026).

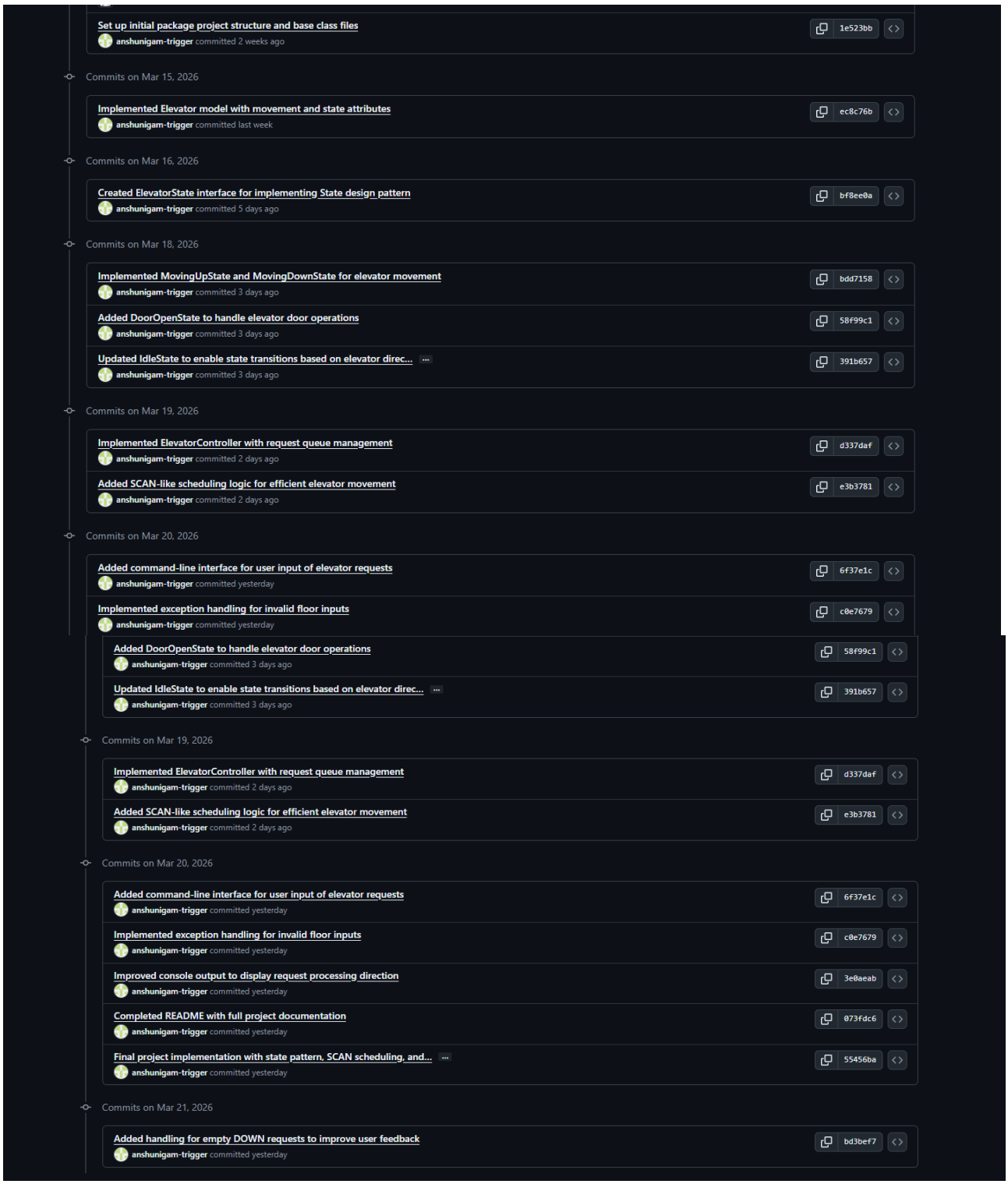

Figure 2: GitHub commit history – 14 commits across 6 days

6.2 Development Progression

Each commit represents a self-contained, meaningful step in development:

- 1e523bb | Mar 7 – Set up initial package project structure and base class files.
- ec8c76b | Mar 15 – Implemented Elevator model with movement and state attributes.
- bf8ee0a | Mar 16 – Created ElevatorState interface for implementing State design pattern.
- bdd7158 | Mar 18 – Implemented MovingUpState and MovingDownState for elevator movement.
- 58f99c1 | Mar 18 – Added DoorOpenState to handle elevator door operations.
- 391b657 | Mar 18 – Updated IdleState to enable state transitions based on elevator direction.
- d337daf | Mar 19 – Implemented ElevatorController with request queue management.
- e3b3781 | Mar 19 – Added SCAN-like scheduling logic for efficient elevator movement.
- 6f37e1c | Mar 20 – Added command-line interface for user input of elevator requests.
- c0e7679 | Mar 20 – Implemented exception handling for invalid floor inputs.
- 3e0aeab | Mar 20 – Improved console output to display request processing direction.
- 073fdc6 | Mar 20 – Completed README with full project documentation.
- 55456ba | Mar 20 – Final project implementation with state pattern, SCAN scheduling, and improved design.
- bd3bef7 | Mar 21 – Added handling for empty DOWN requests to improve user feedback.

7. Conclusion & Future Scope

7.1 Conclusion

This project successfully demonstrates the design and simulation of an elevator control system in Java, meeting all stated objectives. The SCAN-based scheduling algorithm built on two ordered TreeSets ensures efficient floor traversal. The State Design Pattern keeps each operational phase encapsulated in a dedicated class, making the system easy to extend without modifying existing code.

The deliberate use of a Direction enum over plain Strings, Thread.sleep() for realistic travel simulation, and dual TreeSets for directional scheduling all reflect considered design choices that improve robustness, readability, and maintainability. The project demonstrates how core OOP principles — abstraction, polymorphism, encapsulation, and exception handling — work together cohesively in a real-world simulation context.

7.2 Future Scope

- Multi-elevator support: Manage multiple cabins with a dispatcher routing requests to the nearest or least-loaded elevator.
- Request object integration: Wire the existing Request class (sourceFloor, destinationFloor) into the controller for more realistic origin-aware dispatching.
- Graphical interface: Replace the CLI with a JavaFX GUI visualising elevator movement in real time.
- Building-aware validation: Use the Building class to drive floor-range validation, replacing the hardcoded limit of 10.
- Emergency state: Add an EmergencyState overriding all scheduling and directing the elevator to floor 0 immediately.
- Metrics logging: Persist request history, average wait times, and total floors travelled to a file or database for analysis.

Appendix

A. Initial Project Proposal

Project Title: Elevator Control System Simulation Using Object-Oriented Programming and State Design Pattern

1. Introduction / Background

Elevators play an important role in modern multi-storey buildings such as offices, hospitals, shopping malls, and universities. Although using an elevator seems simple from a user's perspective, the system behind it must manage many requests from different floors and control the elevator's movement efficiently. This project aims to build a simulation of an elevator control system using object-oriented programming in Java. The system will process elevator requests, track the elevator's current position, and determine how the elevator should move between floors. To make the design cleaner and closer to real system design, the project will also implement the State Design Pattern, which allows the elevator to behave differently depending on its current state such as idle, moving up, moving down, or door open. Through this project, I aim to understand how real-world control systems are structured using object-oriented design principles.

2. Problem Statement

In buildings with multiple floors, elevators often receive many requests from different floors at the same time. If these requests are not handled properly, the elevator may move inefficiently, resulting in longer waiting times and poor system coordination. The challenge is to design a system that can receive elevator requests, organize them properly, and determine the correct direction and movement of the elevator. The system must also manage different operational states of the elevator such as waiting, moving upward, moving downward, and opening doors. This project will simulate how such a system can be implemented using object-oriented programming concepts and structured design patterns.

3. Objectives

The main objectives of this project are:

- To design and implement a simulation of an elevator control system.
- To apply object-oriented programming principles in a real-world scenario.
- To understand how system states affect program behavior using the State Design Pattern.
- To practice modular software design using multiple interacting classes.
- To implement data structures and exception handling in Java.

4. Target Users

This system simulates elevator operations in buildings such as:

- Office buildings
- Hospitals
- Shopping malls

- Educational institutions

Although the project is a simulation, it represents how elevator control systems work in real environments.

5. Core Features

The proposed system will include the following features:

- Request an elevator from a specific floor
- Select destination floors
- Maintain a queue of elevator requests
- Display the elevator's current floor
- Move the elevator up or down depending on requests
- Handle invalid floor numbers using exception handling
- Manage elevator behavior based on different operational states

6. OOP Concepts Used

This project will demonstrate several important object-oriented programming concepts: Encapsulation Each class will manage its own data and behavior. Abstraction Complex system logic will be hidden behind simple interfaces such as requesting a floor or moving the elevator. Inheritance Different types of elevators or components can extend base classes if the system is expanded. Polymorphism Different objects can respond differently to the same method depending on the system state. Exception Handling The system will handle invalid inputs such as incorrect floor numbers. Collections Data structures such as queues will be used to manage elevator requests.

7. Architecture

The system will be designed using multiple interacting classes to ensure modular and maintainable code.

Key classes in the system may include:

- Elevator – Represents the elevator and manages its movement
- ElevatorController – Handles incoming requests and controls elevator behavior
- Request – Stores information about elevator requests
- Floor – Represents different floors in the building
- Building – Represents the overall environment of the system
- ElevatorState (Interface) – Defines behavior for different states
- IdleState, MovingUpState, MovingDownState, DoorOpenState – Concrete state implementations
- Main – Runs the elevator simulation Each class will have a specific responsibility, helping to maintain a clear and organized system structure.

B. Review Mail

PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@iemintern… Mar 5
To You

Dear Anshu Nigam ,

Project: Elevator Control System Simulation Using Object-Oriented Programming and State Design Pattern
Decision: APPROVED_MINOR
Score: 92

Strengths:
The proposal offers a clear and well-defined problem statement, targeting a practical and relatable real-world system (elevator control). It explicitly lists a comprehensive set of features, demonstrating a good understanding of core system functionalities. A significant strength is the explicit mention and planned application of the State Design Pattern, which is highly appropriate for managing elevator operational states and showcases an advanced understanding of OOP design principles beyond basic concepts. The architecture section clearly outlines key classes and their responsibilities, indicating a modular and maintainable design approach. The explicit mention of core OOP concepts like Abstraction, Inheritance, Polymorphism, Encapsulation, Exception Handling, and Collections further solidifies the project's foundation in object-oriented programming.

Improvements:
To further strengthen the proposal, consider elaborating on the specific algorithm or logic that the 'ElevatorController' will employ for processing and prioritizing requests (e.g., will it follow a SCAN/LOOK-like algorithm, or a simpler FIFO approach?). Clarifying the scope regarding the number of elevators (e.g., a single elevator simulation vs. multiple elevators coordinating) would also be beneficial, as this impacts the complexity of the 'ElevatorController'. Additionally, outlining the intended method for user interaction and request input (e.g., command-line interface, basic GUI) would provide a clearer picture of the simulation's user experience.

Regards,
Course Instructor